

# SQL Editor 실습 & Table Editor 활용

## 1. 제약 조건 동작 확인

SQL Editor에서 아래 쿼리들을 실행해서 제약이 실제로 동작하는지 확인합니다.

# Insert Data

```
INSERT INTO public.app_users (username, display_name) VALUES ('kim', '김학생');
```

The screenshot shows a SQL Editor interface with a dark theme. On the left, there is a sidebar with a search bar labeled "Search queries..." and a list of query folders: "SHARED", "FAVORITES", and "PRIVATE (2)". The main editor area has a tab labeled "Untitled query" and contains the following SQL code:

```
1 INSERT INTO public.app_users (username, display_name) VALUES ('kim', '김학생');  
2  
3 |
```

Below the editor, there is a toolbar with tabs for "Results", "Explain", "Chart", and "Export". The "Results" tab is selected, and it displays the message "Success. No rows returned". To the right of the tabs, there are buttons for "Source", "Primary Database", "Role postgres", and a green "Run" button with a keyboard shortcut "Ctrl ↵".

## UNIQUE 위반

-- 같은 username으로 다시 추가 시도 → 오류 발생!

```
INSERT INTO public.app_users (username, display_name) VALUES ('kim', '김이름');
```

-- ERROR: duplicate key value violates unique constraint "app\_users\_username\_key"

The screenshot shows a SQL Editor interface with a dark theme. On the left, there is a sidebar with a search bar and a list of queries under 'PRIVATE (2)', including two 'Untitled query' files. The main editor area shows a single query: `INSERT INTO public.app_users (username, display_name) VALUES ('kim', '김이름');`. Below the editor, there is a 'Results' tab and a 'Run' button. The 'Results' tab is active, displaying an error message: `Failed to run sql query: ERROR: 23505: duplicate key value violates unique constraint "app_users_username_key"`. The detail of the error is: `DETAIL: Key (username)=(kim) already exists.`. The interface also shows database settings like 'Primary Database' and 'Role postgres'.

## NOT NULL 위반

```
INSERT INTO public.app_users (username) VALUES ('noname');  
-- ERROR: null value in column "display_name" violates not-null constraint
```

The screenshot shows a SQL Editor interface with a dark theme. On the left, there is a sidebar with a search bar labeled 'Search queries...' and a list of queries under the heading 'PRIVATE (2)'. Two queries are listed, both labeled 'Untitled query' with a small 'SQL' icon. The main editor area has a tab labeled 'Untitled query' and contains the following SQL code:

```
1 INSERT INTO public.app_users (username) VALUES ('noname');  
2  
3
```

Below the editor, there is a toolbar with buttons for 'Results', 'Explain', 'Chart', and 'Export'. To the right of these buttons are 'Source', 'Primary Database', 'Role postgres', and a green 'Run' button with a keyboard shortcut 'Ctrl + R'. The 'Results' tab is selected, and it displays an error message:

```
Failed to run sql query: ERROR: 23502: null value in column "display_name" of  
relation "app_users" violates not-null constraint  
DETAIL: Failing row contains (41f4d1d9-9772-48bd-aedc-134dc98f36af, noname, null,  
2026-06-17 11:27:52.057086+00).
```

At the bottom right of the results panel, there is a 'Debug with Assistant' button with a dropdown arrow.

## FK 위반: 존재하지 않는 user\_id

```
INSERT INTO public.conversations (user_id, title)
VALUES ('00000000-0000-0000-0000-000000000000', '테스트');
-- ERROR: violates foreign key constraint
```

The screenshot shows a SQL Editor interface with a dark theme. On the left, there is a sidebar with a search bar labeled 'Search queries...' and a list of queries under 'PRIVATE (2)', including two 'Untitled query' entries. The main editor area displays a SQL query: `1 INSERT INTO public.conversations (user_id, title)`, `2 VALUES ('00000000-0000-0000-0000-000000000000', '테스트');`, `3`, and `4`. Below the editor, there is a 'Results' tab selected, showing an error message: `Failed to run sql query: ERROR: 23503: insert or update on table "conversations" violates foreign key constraint "conversations_user_id_fkey"`, `DETAIL: Key (user_id)=(00000000-0000-0000-0000-000000000000) is not present in table "app_users".` The interface also includes buttons for 'Source', 'Primary Database', 'Role postgres', 'Run Ctrl R', and 'Debug with Assistant'.

## CHECK 위반: 허용되지 않은 role

```
INSERT INTO public.messages (conversation_id, role, content)
VALUES ('대화_UUID', 'bot', '안녕');
-- ERROR: violates check constraint
```

The screenshot shows a SQL Editor interface with a dark theme. On the left, there is a sidebar with a search bar and a list of queries under 'PRIVATE (2)', including two 'Untitled query' files. The main editor area displays a SQL query:

```
1 INSERT INTO public.messages (conversation_id, role, content)
2 VALUES ('대화_UUID', 'bot', '안녕');
3
4
```

Below the query, there are tabs for 'Results', 'Explain', 'Chart', and 'Export'. The 'Results' tab is active, showing an error message: 'Failed to run sql query: ERROR: 22P02: invalid input syntax for type uuid: "대화\_UUID"'. The error message is followed by the query text: 'LINE 2: VALUES ('대화\_UUID', 'bot', '안녕');'. A cursor points to the '대화\_UUID' string. At the bottom right of the error message, there is a 'Debug with Assistant' button.

## 2. 데이터 준비 실습

```
INSERT INTO public.app_users (username, display_name) VALUES ('lee', '이학생');
```

The screenshot displays a SQL Editor interface with a dark theme. On the left, a sidebar contains a search bar labeled 'Search queries...' and a list of query categories: 'SHARED', 'FAVORITES', and 'PRIVATE (2)'. The main editor area shows a single query in a tab labeled 'Untitled query':  
1 INSERT INTO public.app\_users (username, display\_name) VALUES ('lee', '이학생');  
2  
3  
Below the query editor, a toolbar includes buttons for 'Results' (selected), 'Explain', 'Chart', and 'Export'. To the right of these buttons are dropdown menus for 'Source', 'Primary Database', and 'Role postgres', followed by a green 'Run' button with a keyboard shortcut 'Ctrl + R'. The bottom status bar indicates the execution result: 'Success. No rows returned'.

```
-- kim의 UUID를 복사  
select * from public.app_users  
where 1=1  
and username = 'kim';
```

The screenshot shows a database query editor interface. At the top, a tab labeled "SQL Untitled query" is active. Below it, the SQL query is entered in a text area:

```
1 select * from public.app_users  
2 where 1=1  
3 and username = 'kim';  
4
```

Below the query editor, there is a toolbar with buttons for "Results", "Explain", "Chart", and "Export". To the right of these buttons are dropdown menus for "Source", "Primary Database", and "Role postgres", followed by a green "Run" button with a keyboard shortcut "Ctrl + R".

Below the toolbar, the results of the query are displayed in a table with the following columns: **id**, **username**, **display\_name**, and **created\_at**.

| id                                   | username | display_name | created_at                    |
|--------------------------------------|----------|--------------|-------------------------------|
| 4c460602-eaf4-4e6e-a1d2-781843690bf7 | kim      | 김학생          | 2026-06-17 11:24:57.807583+00 |

A red box highlights a context menu option "Copy cell content" that appears over the first row of the table.

```
INSERT INTO public.conversations (user_id, title)
VALUES ('복사한-UUID', 'FastAPI 공부')
RETURNING *; -- 저장된 행을 바로 반환
```

 SQL

Untitled query

+

```
1 INSERT INTO public.conversations (user_id, title)
2 VALUES ('4c460602-eaf4-4e6e-a1d2-781843690bf7', 'FastAPI 공부')
3 RETURNING *; -- 저장된 행을 바로 반환
4 |
```

Results Explain Chart Export ▾

⋮

SourcePrimary Database ▾Role postgres ▾Run Ctrl ↵

| id                                   | user_id                              | title      | created_at                    |
|--------------------------------------|--------------------------------------|------------|-------------------------------|
| 9c738635-17d2-433b-b7b4-a11caf43889c | 4c460602-eaf4-4e6e-a1d2-781843690bf7 | FastAPI 공부 | 2026-06-17 11:41:14.071467+00 |

-- 대화방의 UUID를 복사하여 메시지 추가

```
INSERT INTO public.messages (conversation_id, role, content)
```

```
VALUES ('대화방-UUID', 'user', '안녕하세요!');
```

```
INSERT INTO public.messages (conversation_id, role, content)
```

```
VALUES ('대화방-UUID', 'assistant', '안녕하세요! 무엇을 도와드릴까요?');
```

SQL Untitled query

+

```
1 -- 대화방의 UUID를 복사하여 메시지 추가
2 INSERT INTO public.messages (conversation_id, role, content)
3 VALUES ('9c738635-17d2-433b-b7b4-a11caf43889c', 'user', '안녕하세요!');
4 INSERT INTO public.messages (conversation_id, role, content)
5 VALUES ('9c738635-17d2-433b-b7b4-a11caf43889c', 'assistant', '안녕하세요! 무엇을 도와드릴까요?');
6
```

Results Explain Chart Export ▾

⋮

Source

Primary Database ▾

Role postgres ▾

Run Ctrl ↵

Success. No rows returned

### 3. JOIN 실습 — 3개 테이블 연결

```
SELECT
  u.username,
  u.display_name,
  c.title      AS conversation_title,
  m.role,
  m.content,
  m.created_at
FROM public.messages m
JOIN public.conversations c ON c.id = m.conversation_id
JOIN public.app_users    u ON u.id = c.user_id
ORDER BY m.created_at ASC;
```

SQL

Untitled query

+

```
1 SELECT
2   u.username,
3   u.display_name,
4   c.title      AS conversation_title,
5   m.role,
6   m.content,
7   m.created_at
8 FROM public.messages m
9 JOIN public.conversations c ON c.id = m.conversation_id
10 JOIN public.app_users   u ON u.id = c.user_id
11 ORDER BY m.created_at ASC;
12
```

Results

Explain

Chart

Export

Source

Primary Database

Role postgres

Run Ctrl ↵

| username | display_name | conversation_ | role      | content       | created_at                    |
|----------|--------------|---------------|-----------|---------------|-------------------------------|
| kim      | 김학생          | FastAPI 공부    | user      | 안녕하세요!        | 2026-06-17 11:43:35.672329+00 |
| kim      | 김학생          | FastAPI 공부    | assistant | 안녕하세요! 무엇을 도와 | 2026-06-17 11:43:35.672329+00 |

## 4. 집계 실습 — GROUP BY, HAVING

```
-- 사용자별 대화 수
SELECT u.username, COUNT(c.id) AS 대화수
FROM public.app_users u
LEFT JOIN public.conversations c ON c.user_id = u.id
GROUP BY u.id, u.username
ORDER BY 대화수 DESC;
```

SQL

Untitled query

+

```
1 SELECT u.username, COUNT(c.id) AS 대화수
2 FROM public.app_users u
3 LEFT JOIN public.conversations c ON c.user_id = u.id
4 GROUP BY u.id, u.username
5 ORDER BY 대화수 DESC;
6
```

Results Explain Chart Export 

⋮

Source Primary Database 

⌵

Role postgres 

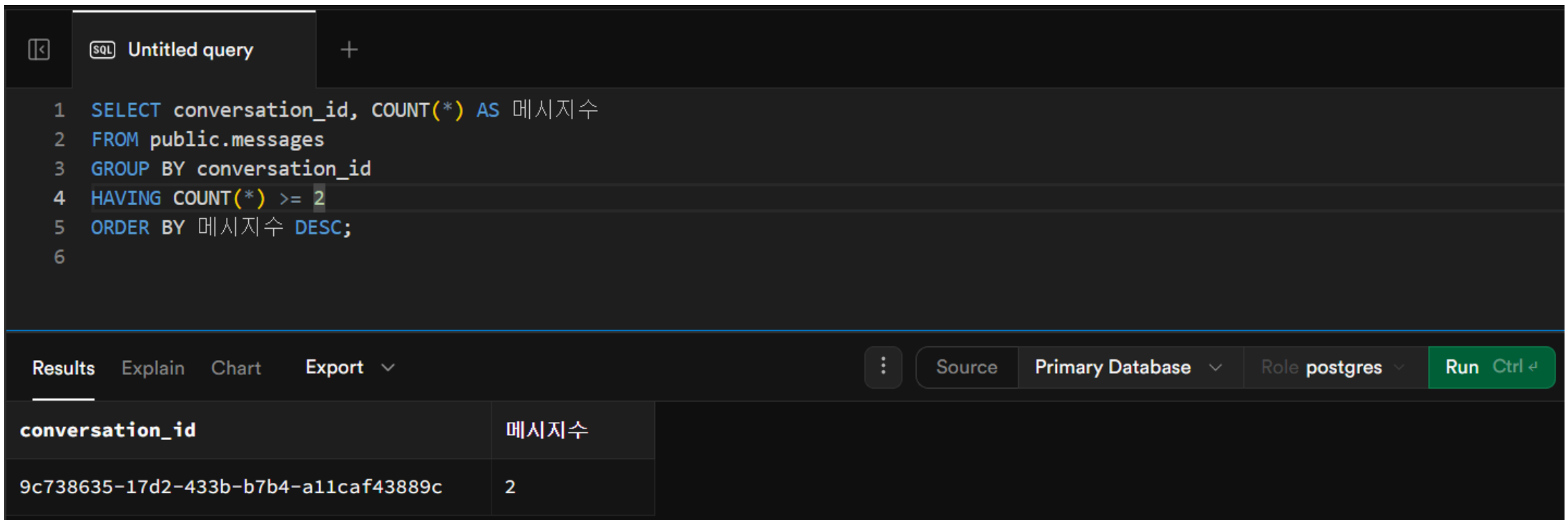
⌵

Run 

⌘

| username | 대화수 |
|----------|-----|
| kim      | 1   |
| lee      | 0   |

```
-- 메시지가 2개 이상인 대화방만 조회 (HAVING - 집계 후 필터)  
SELECT conversation_id, COUNT(*) AS 메시지수  
FROM public.messages  
GROUP BY conversation_id  
HAVING COUNT(*) >= 2  
ORDER BY 메시지수 DESC;
```



The screenshot shows a SQL query editor with a dark theme. The query is entered in a text area and is as follows:

```
1 SELECT conversation_id, COUNT(*) AS 메시지수  
2 FROM public.messages  
3 GROUP BY conversation_id  
4 HAVING COUNT(*) >= 2  
5 ORDER BY 메시지수 DESC;  
6
```

Below the query editor, there is a toolbar with options: Results, Explain, Chart, Export, Source, Primary Database, Role postgres, and a Run button (Ctrl + Enter).

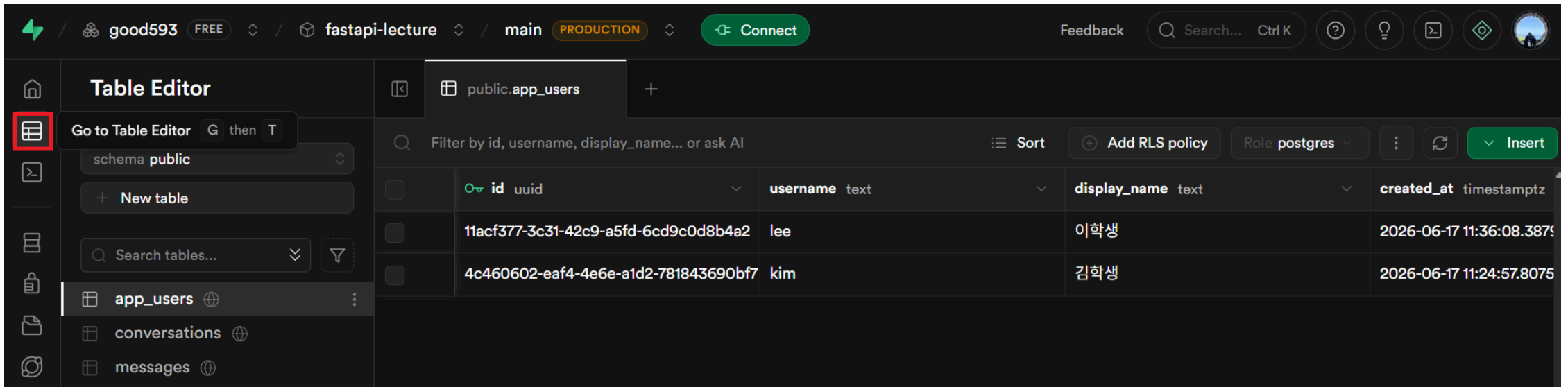
The Results tab is active, displaying a table with the following data:

| conversation_id                      | 메시지수 |
|--------------------------------------|------|
| 9c738635-17d2-433b-b7b4-a11caf43889c | 2    |

## 5. Supabase Table Editor 활용

SQL만으로는 직관적이지 않을 때 Table Editor를 사용합니다.

- 테이블 내용 보기: 좌측 메뉴 → Table Editor → 테이블 선택
- 행 직접 추가/수정/삭제: + Insert row, 셀 클릭 편집, 행 선택 후 Delete
- 필터/정렬: 상단 바의 Filter, Sort 버튼



The screenshot shows the Supabase Table Editor interface. The left sidebar has a 'Table Editor' tab highlighted with a red box. The main area displays the 'public.app\_users' table with the following columns: id (uuid), username (text), display\_name (text), and created\_at (timestampz). The table contains three rows of data. The top row has a checkbox, an 'id' column with a value '11acf377-3c31-42c9-a5fd-6cd9c0d8b4a2', a 'username' column with 'lee', a 'display\_name' column with '이학생', and a 'created\_at' column with '2026-06-17 11:36:08.3879'. The bottom row has a checkbox, an 'id' column with a value '4c460602-eaf4-4e6e-a1d2-781843690bf7', a 'username' column with 'kim', a 'display\_name' column with '김학생', and a 'created\_at' column with '2026-06-17 11:24:57.8075'. The interface includes a search bar, a 'Filter by id, username, display\_name... or ask AI' button, a 'Sort' button, an 'Add RLS policy' button, a 'Role postgres' dropdown, and an 'Insert' button.

|                          | id                                   | username | display_name | created_at               |
|--------------------------|--------------------------------------|----------|--------------|--------------------------|
| <input type="checkbox"/> | 11acf377-3c31-42c9-a5fd-6cd9c0d8b4a2 | lee      | 이학생          | 2026-06-17 11:36:08.3879 |
| <input type="checkbox"/> | 4c460602-eaf4-4e6e-a1d2-781843690bf7 | kim      | 김학생          | 2026-06-17 11:24:57.8075 |